

Proyecto final

David Santiago Vargas Parra – 01240371048

Moises David Baquero Daza – 01240371051

Jose Daniel Nova Muñoz – 01240371059

Docente: Yudi Catherine Diaz Perdomo

Universidad de Santander

Métodos numéricos

2026

1. Resumen

Este proyecto implementa y compara dos métodos numéricos abiertos para la búsqueda de raíces de polinomios: el método de Newton-Raphson y el método de Müller. Se desarrolló una aplicación de escritorio en Python que permite al usuario ingresar un polinomio de cualquier grado mediante sus coeficientes, configurar los parámetros iniciales de cada método y observar en tiempo real el proceso iterativo de convergencia, los errores relativos por iteración, el diagrama de polos en el plano complejo y una comparación de rendimiento entre ambos métodos.

La aplicación integra conceptos de análisis numérico con ingeniería de software, siguiendo una arquitectura modular en cuatro capas: representación del polinomio, lógica de los métodos numéricos, interfaz gráfica y punto de entrada. El producto final es una herramienta educativa y funcional que demuestra cómo los métodos numéricos se aplican en la práctica para resolver ecuaciones polinómicas con raíces reales y complejas.

Palabras clave: Newton-Raphson, Müller, raíces de polinomios, métodos numéricos, convergencia, error relativo.

2. Introducción

En el campo de la ingeniería y las ciencias aplicadas, la resolución de ecuaciones no lineales es una tarea fundamental. En particular, encontrar las raíces de polinomios es un problema recurrente en áreas como el análisis de estabilidad de sistemas de control, el diseño de filtros digitales, la computación gráfica y la modelación matemática. Aunque existen métodos cerrados para polinomios de bajo grado, los métodos numéricos abiertos resultan imprescindibles cuando se trabaja con polinomios de grado elevado o con raíces complejas.

El presente proyecto aborda este problema a través de la implementación computacional de dos métodos numéricos abiertos ampliamente estudiados: Newton-Raphson y Müller. El primero se basa en la linearización de la función mediante su derivada y converge de forma cuadrática cuando la estimación inicial es adecuada. El segundo generaliza la secante a través de una aproximación parabólica, lo que lo hace capaz de encontrar raíces complejas sin necesidad de partir de un valor complejo.

La motivación del proyecto radica en que los métodos numéricos constituyen un puente directo entre las matemáticas y la programación. Construir una herramienta que los implemente y compare permite no solo verificar los conceptos teóricos, sino también entender sus fortalezas, limitaciones y comportamiento en distintos escenarios prácticos.

3. Objetivos

3.1 Objetivo General

Desarrollar una aplicación de escritorio en Python que implemente y compare los métodos numéricos abiertos Newton-Raphson y Müller para la búsqueda de raíces de polinomios, visualizando el proceso iterativo y el comportamiento de convergencia de cada método.

3.2 Objetivos Específicos

1. Investigar y documentar los fundamentos matemáticos de los métodos de Newton-Raphson y Müller, incluyendo su derivación, criterios de convergencia y casos de fallo.
2. Diseñar una arquitectura modular orientada a objetos que separe la representación del polinomio, la lógica de los métodos numéricos y la interfaz gráfica de usuario.
3. Implementar los algoritmos de Newton-Raphson y Müller con soporte para números complejos, control de tolerancia y límite de iteraciones.
4. Construir una interfaz gráfica con Tkinter que permita ingresar parámetros, ejecutar ambos métodos y visualizar los resultados en tablas comparativas y diagrama de polos.
5. Evaluar y comparar el rendimiento de ambos métodos en términos de número de iteraciones, error final y tiempo de cómputo para distintos polinomios de prueba.

4. Justificación y Planteamiento del Problema

4.1 Planteamiento del Problema

En los cursos de métodos numéricos, uno de los desafíos más frecuentes es la brecha entre la comprensión teórica de los algoritmos y su verificación práctica. Los estudiantes aprenden las fórmulas iterativas de Newton-Raphson y Müller, pero rara vez tienen la oportunidad de observar cómo evoluciona el error en cada iteración, cómo se comportan ante raíces complejas o cómo difieren en eficiencia para distintos polinomios.

Actualmente, muchas herramientas educativas disponibles son de propósito general (como MATLAB o Wolfram Alpha) y no permiten visualizar el proceso interno de los algoritmos de forma pedagógica. La ausencia de una herramienta educativa gratuita, específica para métodos abiertos y con interfaz accesible, dificulta el aprendizaje comparativo y la experimentación rápida por parte de los estudiantes.

4.2 Justificación

Desde una perspectiva práctica, el proyecto resuelve la necesidad de contar con una herramienta educativa que permita experimentar con ambos métodos, observar la convergencia, detectar casos de divergencia y comparar su comportamiento sin necesidad de configurar entornos complejos.

Desde una perspectiva académica, el proyecto aplica directamente conceptos de análisis numérico, álgebra de números complejos, derivación simbólica y teoría de polinomios, todos ellos fundamentales en la formación de un ingeniero. La implementación refuerza el aprendizaje

activo al exigir una comprensión profunda de los algoritmos para traducirlos en código funcional.

Desde una perspectiva tecnológica, el proyecto demuestra cómo Python, con sus bibliotecas NumPy, SymPy y Matplotlib, puede emplearse para construir herramientas científicas robustas de escritorio, lo que resulta de valor directo en el contexto de la ingeniería de software aplicada.

5. Marco de Referencia

5.1 Marco Teórico

5.1.1 Polinomios y Evaluación Numérica

Un polinomio de grado n se define como:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0, \quad a_n \neq 0$$

donde los coeficientes $a_i \in \mathbb{R}$ (o $\mathbb{C}$) y x puede tomar valores reales o complejos. Las raíces del polinomio son los valores de x para los cuales $P(x) = 0$. Por el Teorema Fundamental del Álgebra, un polinomio de grado n tiene exactamente n raíces en $\mathbb{C}$ (contadas con multiplicidad).

5.1.2 Método de Newton-Raphson

El método de Newton-Raphson es un algoritmo iterativo de un punto para encontrar raíces de funciones diferenciables. Partiendo de una estimación inicial x_0 , la fórmula de iteración se obtiene aproximando la función por su recta tangente en el punto actual:

$$x_{i+1} = x_i - P(x_i) / P'(x_i)$$

El método presenta convergencia cuadrática cuando la estimación inicial está suficientemente cerca de la raíz y $P'(x_i) \neq 0$. El criterio de parada utilizado en la implementación es el error relativo porcentual:

$$\varepsilon_a = |(x_{i+1} - x_i) / x_{i+1}| \times 100\% < \text{tolerancia}$$

El método puede fallar si la derivada se anula en algún punto de la iteración, si la estimación inicial está lejos de la raíz, o si el polinomio tiene raíces múltiples que ralentizan la convergencia. En la implementación se captura el caso $P'(x_i) = 0$ y se retorna un mensaje de diagnóstico.

5.1.3 Método de Müller

El método de Müller es una extensión del método de la secante que utiliza tres puntos iniciales x_0, x_1, x_2 para ajustar una parábola (polinomio de grado 2) a través de los puntos $(x_0, P(x_0))$, $(x_1, P(x_1))$ y $(x_2, P(x_2))$. La siguiente estimación de la raíz se obtiene como la raíz de esa parábola más cercana a x_2 :

$$\begin{aligned} h_1 &= x_1 - x_0, & h_2 &= x_2 - x_1 \\ \delta_1 &= (P(x_1) - P(x_0)) / h_1, & \delta_2 &= (P(x_2) - P(x_1)) / h_2 \\ a &= (\delta_2 - \delta_1) / (h_2 + h_1), & b &= a \cdot h_2 + \delta_2, & c &= P(x_2) \\ x_3 &= x_2 - 2c / (b \pm \sqrt{b^2 - 4ac}) \end{aligned}$$

Se elige el signo del denominador que maximice su valor absoluto para mayor estabilidad numérica. La principal ventaja del método de Müller frente a Newton-Raphson es su capacidad intrínseca de encontrar raíces complejas incluso partiendo de puntos iniciales reales, gracias al uso de aritmética compleja en el discriminante.

5.2 Marco Conceptual

A continuación, se definen los términos clave del proyecto:

Raíz de un polinomio: valor $x_0 \in \mathbb{C}$ tal que $P(x_0) = 0$. Corresponde geométricamente al punto de corte de la curva con el eje horizontal en el plano real, o a un polo en el plano complejo.

Convergencia: propiedad de una sucesión iterativa x_n de acercarse a un límite (la raíz) a medida que $n \rightarrow \infty$. La velocidad de convergencia puede ser lineal, cuadrática, etc.

Error relativo porcentual: medida del cambio relativo entre dos iteraciones consecutivas. Se usa como criterio de parada: $\varepsilon_a = |(x_{n+1} - x_n) / x_{n+1}| \times 100\%$.

Tolerancia: umbral es prefijado por el usuario. El método se detiene cuando $\varepsilon_a < \varepsilon_s$.

Diagrama de polos: representación gráfica en el plano complejo (parte real vs. parte imaginaria) de las raíces halladas. Permite identificar raíces conjugadas y la ubicación respecto al origen.

Polinomio de Taylor: base teórica de Newton-Raphson; la recta tangente es el polinomio de Taylor de primer orden de P en x_0 .

Número complejo: número de la forma $a + bi$, donde a es la parte real, b la parte imaginaria e $i = \sqrt{-1}$. Python maneja números complejos de forma nativa con el tipo `complex`.

5.3 Marco Tecnológico

Herramienta	Versión	Uso en el Proyecto
Python 3.11	3.11.x	Lenguaje principal. Elegido por su soporte nativo de números complejos, su ecosistema científico y su claridad sintáctica.
NumPy	1.26.4	Evaluación vectoriales de polinomios mediante <code>np.poly1d</code> . Permite evaluar $P(x)$ de forma eficiente para valores reales y complejos.
SymPy	1.12	Derivación simbólica del polinomio. Se usa para calcular $P'(x)$ de forma exacta, extrayendo los coeficientes de la derivada.

Herramienta	Versión	Uso en el Proyecto
Matplotlib	3.8.4	Generación del diagrama de polos en el plano complejo, integrado en la interfaz mediante FigureCanvasTkAgg.
Tkinter	Estándar	Biblioteca estándar de Python para interfaces gráficas de escritorio. Usada para construir todos los widgets, tablas y la ventana principal.
VS Code / PyCharm	Reciente	Entornos de desarrollo integrado usados para escribir, depurar y organizar el código fuente del proyecto.
Git / GitHub	Reciente	Control de versiones y repositorio del proyecto. Permite trabajo colaborativo y trazabilidad de cambios.

6. Marco Metodológico

6.1 Tipo de Investigación

El proyecto corresponde a una investigación aplicada de enfoque cuantitativo y alcance descriptivo-experimental. Se emplean métodos numéricos existentes (Newton-Raphson y Müller) para construir una herramienta de software concreta, y se mide su comportamiento mediante variables cuantitativas: número de iteraciones, error relativo y tiempo de ejecución.

6.2 Metodología de Desarrollo

Se adoptó la metodología de Prototipado Evolutivo, apropiada dado que el sistema involucra una interfaz gráfica cuyo diseño final no era completamente evidente al inicio. El proceso siguió ciclos cortos de construcción, prueba y refinamiento:

Prototipo 1 — Lógica núcleo: se implementaron las clases Polinomio, NewtonRaphson y Muller sin interfaz gráfica, ejecutando pruebas desde consola con polinomios conocidos para verificar la corrección de los algoritmos.

Prototipo 2 — Visualización y refinamiento: se integró el diagrama de polos con Matplotlib, se añadieron mensajes de diagnóstico, manejo de excepciones y el reporte de tiempo de ejecución. Se realizaron pruebas con distintos polinomios y se corrigieron comportamientos inesperados.

6.3 Fases del Proyecto

Fase	Duración	Actividades principales
Fase 1 — Investigación teórica	Semana 1	Estudio de Newton-Raphson, Müller, aritmética compleja
Fase 2 — Diseño de arquitectura	Semana 2	Definición de clases, módulos y contratos de interfaz
Fase 3 — Implementación (Prototipo 1)	Semana 3	Clases Polinomio, MetodoNumerico, NewtonRaphson, Muller
Fase 6 — Pruebas y documentación (Terminación con el prototipo 2)	Semana 4	Casos de prueba, correcciones finales, redacción del informe

7. Alcance del Proyecto

7.1 Lo que incluye el proyecto

- Soporte para polinomios de cualquier grado ≥ 1 , ingresados mediante sus coeficientes en orden descendente.
- Implementación del método Newton-Raphson con derivada simbólica automática, soporte para punto inicial complejo y criterio de parada por error relativo.
- Implementación del método de Müller con soporte de aritmética compleja nativa, tres puntos iniciales configurables y criterio de parada por error relativo.
- Interfaz gráfica de escritorio con tablas de iteraciones comparativas para ambos métodos.
- Diagrama de polos en el plano complejo con las raíces halladas por cada método.
- Reporte de número de iteraciones, error final y tiempo de ejecución en milisegundos.
- Manejo de casos de fallo: derivada cero (Newton-Raphson), denominador cero (Müller) y no convergencia dentro del límite de iteraciones.

7.2 Lo que NO incluye el proyecto

- No se implementan métodos cerrados (bisección, falsa posición, punto fijo, secante).
- No se buscan todas las raíces del polinomio de forma simultánea ni automática (deflación de polinomios).
- No se incluye exportación de resultados a archivos (CSV, Excel, PDF).
- No se despliega la aplicación en entorno web ni en la nube.

- No se soportan funciones trascendentes (sen, cos, e^x); únicamente polinomios.
- No se incluye un sistema de autenticación ni base de datos.

8. Requerimientos del Sistema

8.1 Requerimientos Funcionales

ID	Descripción	Prioridad
RF-01	El sistema debe permitir al usuario ingresar los coeficientes de un polinomio separados por comas.	Alta
RF-02	El sistema debe calcular raíces del polinomio usando el método Newton-Raphson dado un punto inicial x_0 .	Alta
RF-03	El sistema debe calcular raíces del polinomio usando el método de Müller dados tres puntos iniciales x_0, x_1, x_2 .	Alta
RF-04	El sistema debe mostrar en tabla las iteraciones de cada método: $n, x_n, f(x_n)$, error relativo.	Alta
RF-05	El sistema debe visualizar un diagrama de polos en el plano complejo con las raíces halladas.	Media
RF-06	El sistema debe permitir configurar tolerancia y número máximo de iteraciones.	Media
RF-07	El sistema debe reportar si el método convergió o no, y el mensaje de diagnóstico.	Alta

ID	Descripción	Prioridad
RF-08	El sistema debe calcular y mostrar el tiempo de ejecución de cada método en milisegundos.	Baja

8.2 Requerimientos No Funcionales

ID	Descripción	Categoría
RNF-01	El sistema debe calcular la raíz de un polinomio de grado ≤ 10 en menos de 2 segundos.	Rendimiento
RNF-02	La interfaz debe ser intuitiva y no requerir manual de usuario para funciones básicas.	Usabilidad
RNF-03	El sistema debe funcionar en Windows 10, Windows 11, macOS y Linux con Python 3.10+.	Portabilidad
RNF-04	El código debe estar modularizado en al menos 4 archivos con responsabilidades separadas.	Mantenibilidad
RNF-05	El sistema debe manejar excepciones (derivada cero, denominador cero) sin cerrarse.	Robustez

9. Beneficios e Impacto Esperado

El desarrollo de esta herramienta genera un impacto positivo en varios frentes. A nivel académico, permite que los estudiantes de métodos numéricos dejen de limitarse a memorizar fórmulas y puedan observar, en tiempo real, cómo evoluciona el error en cada iteración, cómo difieren las velocidades de convergencia entre Newton-Raphson y Müller, y cómo se distribuyen las raíces en el plano complejo, lo que convierte un concepto abstracto en algo visualmente comprensible. A nivel técnico, la arquitectura modular del sistema no es un detalle menor: al separar claramente la capa de dominio, la lógica de los métodos y la presentación, el código queda abierto a crecer sin romperse, y añadir un nuevo método numérico en el futuro sería tan sencillo como crear una nueva clase que herede de `MetodoNumerico`, sin tocar nada de lo que ya funciona. Para el usuario directo, la herramienta elimina el trabajo manual de calcular iteraciones a mano o en una hoja de cálculo, detecta automáticamente cuándo un método falla y por qué, y produce el diagrama de polos que de otra forma requeriría graficar a mano números complejos. Y aunque el alcance actual es educativo, la estructura del sistema es exactamente la misma que se usa en aplicaciones de ingeniería real, como el análisis de estabilidad de sistemas de control o el diseño de filtros digitales, lo que demuestra que Python, con las bibliotecas adecuadas, es una alternativa real y gratuita a herramientas comerciales como MATLAB.

10. Recomendaciones para Versiones Futuras

- Implementar deflación de polinomios para hallar todas las raíces de forma secuencial: una vez encontrada una raíz r , dividir el polinomio entre $(x - r)$ y aplicar el método al cociente.

- Añadir soporte para el método de Bairstow, que extrae directamente factores cuadráticos reales de polinomios con raíces complejas conjugadas, evitando la aritmética compleja.
- Implementar la representación gráfica de $P(x)$ en el eje real junto con la visualización de las aproximaciones iterativas sucesivas, para hacer más pedagógico el proceso de Newton-Raphson.

11. Referencias Bibliográficas

- o Chapra, S. C., & Canale, R. P. (2010). *Métodos numéricos para ingenieros* (6.^a ed.). McGraw-Hill. Disponible en:
<https://pdfcoffee.com/metodos-numericos-para-ing-6a-chapra-9-pdf-free.html>
- o Muller, D. E. (1956). A method for solving algebraic equations using an automatic computer. *Mathematics of Computation*, 10(56), 208–215. Disponible en:
<https://www.ams.org/journals/mcom/1956-10-056/S0025-5718-1956-0083822-0/S0025-5718-1956-0083822-0.pdf>
- o Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
<https://doi.org/10.1038/s41586-020-2649-2>
- o Python Software Foundation. (2024). *Python 3.11 documentation*.
<https://docs.python.org/3.11/>

12. Apéndice

Apéndice A — Código fuente relevante

A.1 — Algoritmo Newton-Raphson

```
javascriptfunction runNewton() {  
  
    const x0    = parseFloat(document.getElementById('n-x0').value);  
  
    const tol   = parseFloat(document.getElementById('n-tol').value);  
  
    const maxIter= parseInt(document.getElementById('n-maxiter').value);
```

```

const h    = parseFloat(document.getElementById('n-h').value);

const Tmax = parseFloat(document.getElementById('n-tmax').value);


// Interpolación lineal de T(q) con los datos ingresados

function T(q) { /* interpola el tiempo de respuesta */ }


//  $f(q) = T(q) - T_{max}$ 

function f(q) { return T(q) - Tmax; }


// Derivada numérica por diferencias centradas

function df(q) { return (f(q + h) - f(q - h)) / (2 * h); }


let x = x0;

for (let i = 0; i < maxIter; i++) {

    const fx = f(x);

    const dfx = df(x);

    const xNew = x - fx / dfx;

    const err = Math.abs((xNew - x) / xNew);

    if (err < tol) break;

    x = xNew;

}

}

```

A.2 — Algoritmo Método de Müller

```
javascriptfunction runMuller() {  
    // Ajuste polinomial sobre  $g(t) = \Delta T(t)$   
    // Construcción de parábola con 3 puntos:  
    //  $p(x) = a(x-x_2)^2 + b(x-x_2) + c$   
  
    function mullerStep(x0, x1, x2) {  
        const f0 = polyEval(coeffs, x0);  
        const f1 = polyEval(coeffs, x1);  
        const f2 = polyEval(coeffs, x2);  
  
        const h0 = x1 - x0, h1 = x2 - x1;  
        const d0 = (f1 - f0) / h0, d1 = (f2 - f1) / h1;  
        const a = (d1 - d0) / (h1 + h0);  
        const b = a * h1 + d1;  
        const c = f2;  
  
        // Discriminante — puede ser negativo (raíz compleja)  
        const disc = complexSqrt({ re: b*b - 4*a*c, im: 0 });  
        const denom = complexAbs(complexAdd({re:b,im:0}, disc)) >  
            complexAbs(complexSub({re:b,im:0}, disc))  
            ? complexAdd({re:b,im:0}, disc)
```

```

: complexSub({re:b,im:0}, disc);

return complexSub({re:x2,im:0},
    complexDiv({re:2*c,im:0}, denom));
}
}

```

Apéndice C — Glosario de términos matemáticos e informáticos

Término	Definición
q*	Punto de saturación: número de consultas concurrentes donde $T(q) = T_{\max}$
SLA	Service Level Agreement — acuerdo de nivel de servicio que define $T_{\max}$
T(q)	Función de tiempo de respuesta en función de consultas concurrentes
T_{max}	Umbral máximo de tiempo de respuesta aceptable (en milisegundos)
g(t) = ΔT(t)	Variación del tiempo de respuesta a lo largo del tiempo o versiones
ε (épsilon)	Tolerancia de error relativo para criterio de parada de la iteración

h	Paso para el cálculo de la derivada numérica por diferencias centradas
Convergencia cuadrática	Tipo de convergencia donde los dígitos correctos se duplican por iteración
Convergencia superlineal	Convergencia de orden ≈ 1.84 , más lenta que cuadrática pero sin necesitar derivada
Raíz compleja	Raíz con parte imaginaria; en este contexto indica oscilación inestable en el sistema
Raíz real	Raíz sin parte imaginaria; indica un punto de transición en la variación del sistema
Diferencias centradas	Método numérico para aproximar $f'(x)$ usando $f(x+h)$ y $f(x-h)$
Interpolación lineal	Estimación de $T(q)$ entre dos puntos conocidos de los datos de entrada
Polinomio de ajuste	Polinomio de grado n que aproxima la serie temporal $g(t)$ para aplicar Müller
Chart.js	Librería JavaScript de código abierto para generar gráficas interactivas
CDN	Content Delivery Network — red para cargar librerías externas sin instalarlas

Apéndice D — Capturas de pantalla del sistema funcionando

D.1 — Vista general de la interfaz Panel principal con las 4 pestañas: Newton-Raphson, Müller, Comparativa y Teoría. Fondo oscuro con grilla decorativa verde semitransparente.

The screenshot shows the EquilibraDB interface with the Newton-Raphson method selected. The interface is dark-themed with a green grid background. The top bar includes the logo 'EQ EquilibraDB' and the subtitle 'Balanceador numérico de carga en bases de datos · Métodos Numéricos · Ingeniería de Software'. The main content area has four tabs: Newton-Raphson, Müller, Comparativa, and Teoría. A text box explains the module's purpose: '¿Qué hace este módulo? Aplica Newton-Raphson para encontrar el punto de saturación q^* el número exacto de consultas concurrentes donde el tiempo de respuesta de tu base de datos supera el umbral aceptable (SLA). $f(q) = T(q) - T_{max}$. La raíz $f(q^*) = 0$ es el límite de carga.'

The interface is divided into two main sections: 'PARAMETROS DEL MÉTODO' and 'DATOS DE RENDIMIENTO'.

PARAMETROS DEL MÉTODO

- Punto inicial x_0 (consultas concurrentes): 50
- Tolerancia ϵ (error relativo mínimo): 0.0001
- Máximo de iteraciones: 50
- Paso h para derivada numérica: 0.001

DATOS DE RENDIMIENTO

- Tiempo límite aceptable T_{max} (ms): 200
- Datos CSV: consultas, tiempo_ms (una fila por línea)
10, 45
20, 82
30, 85
40, 115
50, 150
- Formato: consultas_concurrentes, tiempo_respuesta_ms

Buttons: 'Ejecutar Newton' (green) and 'Cargar ejemplo' (grey).

GRÁFICA $F(q)$ Y CONVERGENCIA

The graph area is currently empty, showing the convergence of the Newton-Raphson method.

D.2 — Panel Newton-Raphson ejecutado Muestra el formulario con parámetros ingresados, la tabla de iteraciones con columnas: Iter, x_n , $f(x_n)$, $f'(x_n)$, x_{n+1} , Error rel., Estado. La fila de convergencia aparece resaltada en verde con marca ✓. La gráfica muestra la curva $T(q)$, la línea horizontal de T_{max} y el punto q^* marcado.



Newton-Raphson

Müller

Comparativa

Teoría

¿Qué hace este módulo? Aplica Newton-Raphson para encontrar el punto de saturación q^* , el número exacto de consultas concurrentes donde el tiempo de respuesta de tu base de datos supera el umbral aceptable (SLA). $f(q) = T(q) - T_{max}$. La raíz $f(q^*) = 0$ es el límite de carga.

PARAMETROS DEL MÉTODO

Punto inicial x_0 (consultas concurrentes)

50

Tolerancia ϵ (error relativo mínimo)

0.0001

Máximo de iteraciones

50

Paso h para derivada numérica

0.001

DATOS DE RENDIMIENTO

Tiempo límite aceptable T_{max} (ms)

200

Datos CSV: consultas, tiempo_ms (una fila por línea)

10, 45
20, 62
30, 85
40, 115
50, 155

Formato: consultas_concurrentes, tiempo_respuesta_ms

✓ Convergencia alcanzada

El punto de saturación de la base de datos es $q^* \approx 56.77$ consultas concurrentes. Con más de 57 usuarios simultáneos, el tiempo de respuesta supera los 200 ms (SLA definido).

Convergió en 3 iteraciones con error $7.509e-14$.

RAÍZ q^*

56.77

consultas concurrentes

ITERACIONES

3

hasta convergencia

ERROR FINAL

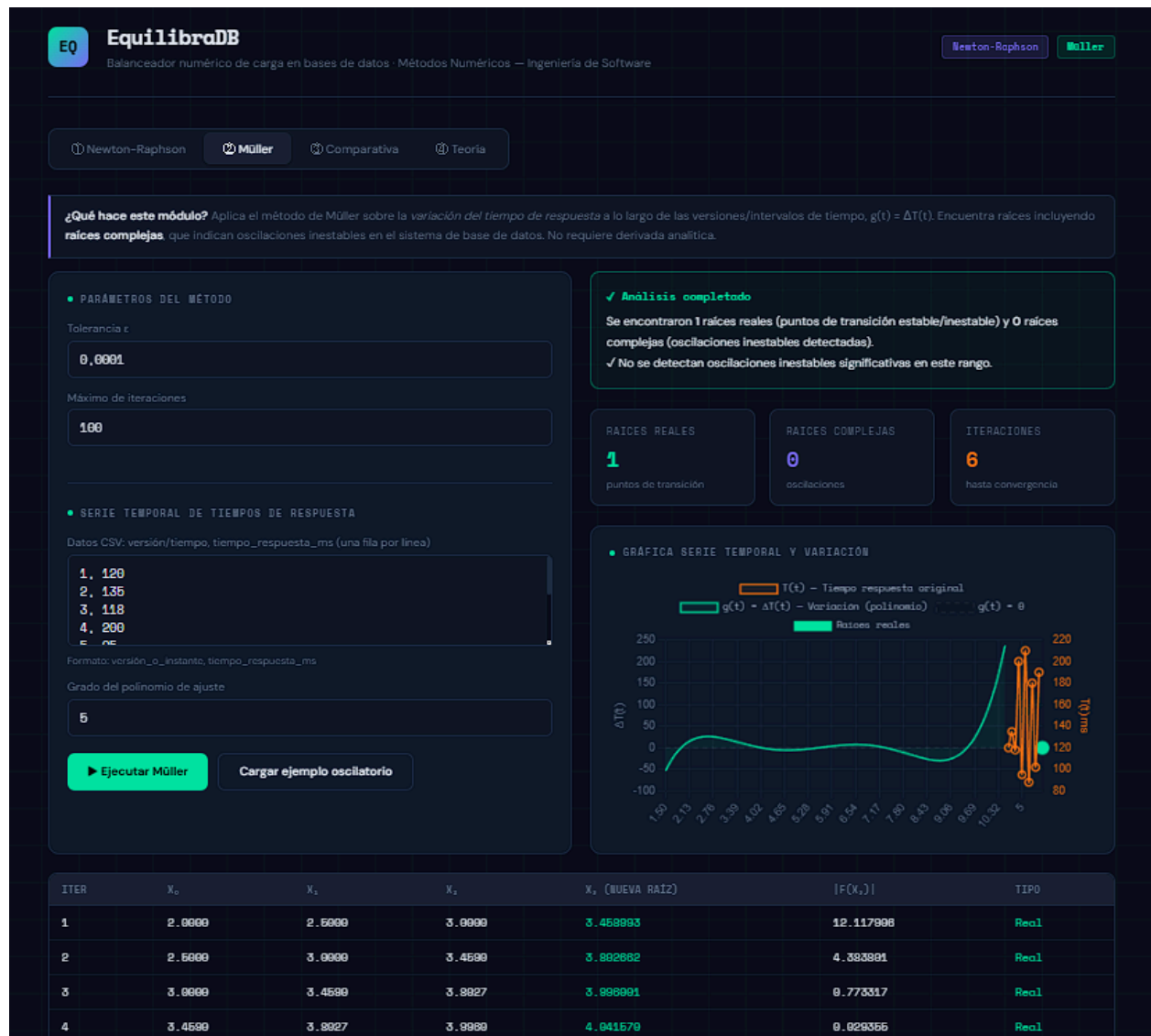
7.51e-14

error relativo

GRAFICA $F(q)$ Y CONVERGENCIA



D.3 — Panel Müller ejecutado Muestra las tarjetas de resumen con: número de raíces reales, número de raíces complejas e iteraciones totales. La gráfica presenta la serie $T(t)$ original en naranja, el polinomio $g(t)$ en verde y los puntos de raíces reales marcados.



D.4 — Panel Comparativa Vista lado a lado de Newton vs Müller con barras de progreso para iteraciones y error. Gráfica de convergencia comparada mostrando $\log_{10}(\text{error})$ por iteración para ambos métodos. Bloque de conclusión automática generado al final.

